



Carátula para entrega de prácticas

Laboratorio de computación salas A y B

Profesor: M.C Rene Adrian Davila Perez

Asignatura: Programación orientada a objetos

Grupo: 7

*No de
Práctica(s):* Etapa 1 Proyecto

Integrante(s): 323036285
323279866
323209432
323161891
323063609

*No. De Lista o
brigada* 3

Semestre: 2027-1

*Fecha de
entrega:* 28/ 09 / 2026

Obervaciones:

CALIFICACIÓN: _____

Índice

1. Introducción	2
2. Marco Teórico	3
2.1. Programación Orientada a Objetos	3
2.1.1. Clases y Objetos	3
2.1.2. Atributos y Métodos	3
2.1.3. Constructores	3
2.2. Estructuras de Datos	4
2.2.1. ArrayList	4
2.2.2. HashMap	4
2.2.3. TreeSet	5
2.3. Colecciones	5
2.3.1. Map	5
2.3.2. Set	5
2.4. Métodos de las Colecciones	5
2.4.1. ArrayList	5
2.4.2. HashMap	5
2.4.3. TreeSet	5
2.5. Métodos de las Cadenas	5
2.6. Ciclos y Estructuras de Control	6
2.6.1. Ciclo for	6
2.6.2. Ciclo while	6
2.6.3. Ciclo for-each	6
2.6.4. Estructura if-else	6
2.7. Validaciones y Precondiciones	6
2.8. Importaciones Utilizadas	6
2.9. Conceptos Teóricos Usados	7
2.9.1. Métodos Sobrecargados	7
3. Desarrollo	8
4. Resultados	18
5. Conclusiones	21

1. Introducción

Al iniciar en un nuevo lenguaje de programación, al estudiantado se le plantea el reto de desarrollar un proyecto final para evaluar sus conocimientos, éstos proyectos permiten múltiples soluciones, lo cuál se vuelve un problema a la hora de programar. Además, gracias a su extensión, se requiere dar avances para tener un desarrollo ordenado.

Resolver estos problemas nos ayudará a demostrar lo que hemos aprendido a lo largo del curso acerca de la programación orientada a objetos, y lograr avances en el proyecto final.

Objetivos

- Comenzar con el proyecto final, para que el alumnado pueda demostrar lo que ha aprendido a lo largo de este periodo.

2. Marco Teórico

Para la implementación del sistema hospitalario se emplearon conceptos de la programación orientada a objetos para el registro y asignación de los pacientes, médicos y enfermeros. A continuación se detallan los conceptos aplicados:

2.1. Programación Orientada a Objetos

La Programación Orientada a Objetos tiene varias cosas fundamentales, como:

2.1.1. Clases y Objetos

Una clase es un molde o plantilla que define atributos y métodos. Un objeto es una instancia de una clase y permiten encapsular datos y métodos en una unidad cohesiva [7].

En el proyecto se implementaron cuatro clases:

- **Clase Paciente:** Encapsula los datos y comportamiento de los pacientes del hospital.
- **Clase Médico:** Encapsula los datos y comportamiento del personal médico y su lista de pacientes asignados.
- **Clase Enfermero:** Encapsula los datos y comportamiento del personal de enfermería y el tratamiento de los pacientes.
- **Clase Sistema:** Es el cerebro del sistema hospitalario.

2.1.2. Atributos y Métodos

Los atributos almacenan el estado del objeto. Los métodos definen el comportamiento del objeto y en este proyecto se utilizaron atributos de tipos primitivos (`int`, `char`, `String`) y referencias a objetos [7].

2.1.3. Constructores

Un constructor es un método especial que se usa cuando se crea un objeto y su propósito es inicializarlo en un estado válido [2]. En el proyecto, cada clase implementó un constructor:

- `Paciente(String nombre, String apellidos, String especialidad)`
- `Médico(String nombre, String especialidad, int cédula)`
- `Enfermero(String nombre, String especialidad, int cédula)`
- `Sistema(String nombreHospital)`

2.2. Estructuras de Datos

2.2.1. ArrayList

`ArrayList` es una estructura de datos de tamaño dinámico que permite almacenar elementos en orden y acceder a ellos con índices [4].

Características:

- Crece o decrece dinámicamente.
- Permite duplicados.

Se usó para listas de médicos, enfermeros, pacientes activos y pacientes curados.

2.2.2. HashMap

`HashMap` es una estructura de pares clave-valor que usa una tabla hash [5]. Características:

- No puede haber duplicados de claves.
- Los valores sí pueden repetirse.
- No tienen orden específico.

Se utilizó en `verListaPacientes()` de médicos y enfermeros para asociar información y luego ordenarla con `TreeSet`.

2.2.3. TreeSet

`TreeSet` almacena elementos únicos y ordenados ascendentemente mediante un árbol binario balanceado [6]. Características:

- No permite duplicados.
- Mantiene elementos ordenados alfabéticamente.

Se usó para ordenar nombres de pacientes y evitar duplicados.

2.3. Colecciones

2.3.1. Map

Usa pares clave-valor y métodos como `put()`, `get()`, `containsKey()`, `keySet()` y `remove()` [5].

2.3.2. Set

Colección sin duplicados, implementada mediante árbol binario [ref5].

2.4. Métodos de las Colecciones

2.4.1. ArrayList

`add()`, `get()`, `remove()`, `size()`, `isEmpty()`.

2.4.2. HashMap

`put()`, `get()`, `keySet()`.

2.4.3. TreeSet

`add()`.

2.5. Métodos de las Cadenas

`toLowerCase()`: Convierte caracteres a minúsculas para comparaciones insensibles [3].

2.6. Ciclos y Estructuras de Control

2.6.1. Ciclo for

Itera sobre listas y posiciones específicas de `ArrayList`, además de ciclos anidados [8].

2.6.2. Ciclo while

Ejecuta mientras la condición sea verdadera, usado para procesar pacientes [8].

2.6.3. Ciclo for-each

Itera sobre colecciones sin índices, usado en `keySet()` y `TreeSet` [1].

2.6.4. Estructura if-else

Permite validar referencias nulas, capacidad de médicos/enfermeros y estado de pacientes [8].

2.7. Validaciones y Precondiciones

Se usaron condicionales `if` para validar referencias nulas y estados de pacientes (E, C, T, A).

2.8. Importaciones Utilizadas

- `java.util.ArrayList`
- `java.util.HashMap`
- `java.util.TreeSet`
- `java.util.Set`
- `java.util.Map`
- `java.util.Collection`
- `java.util.Scanner` [9]

2.9. Conceptos Teóricos Usados

2.9.1. Métodos Sobrecargados

El método `asignarPaciente()` fue implementado en dos versiones:

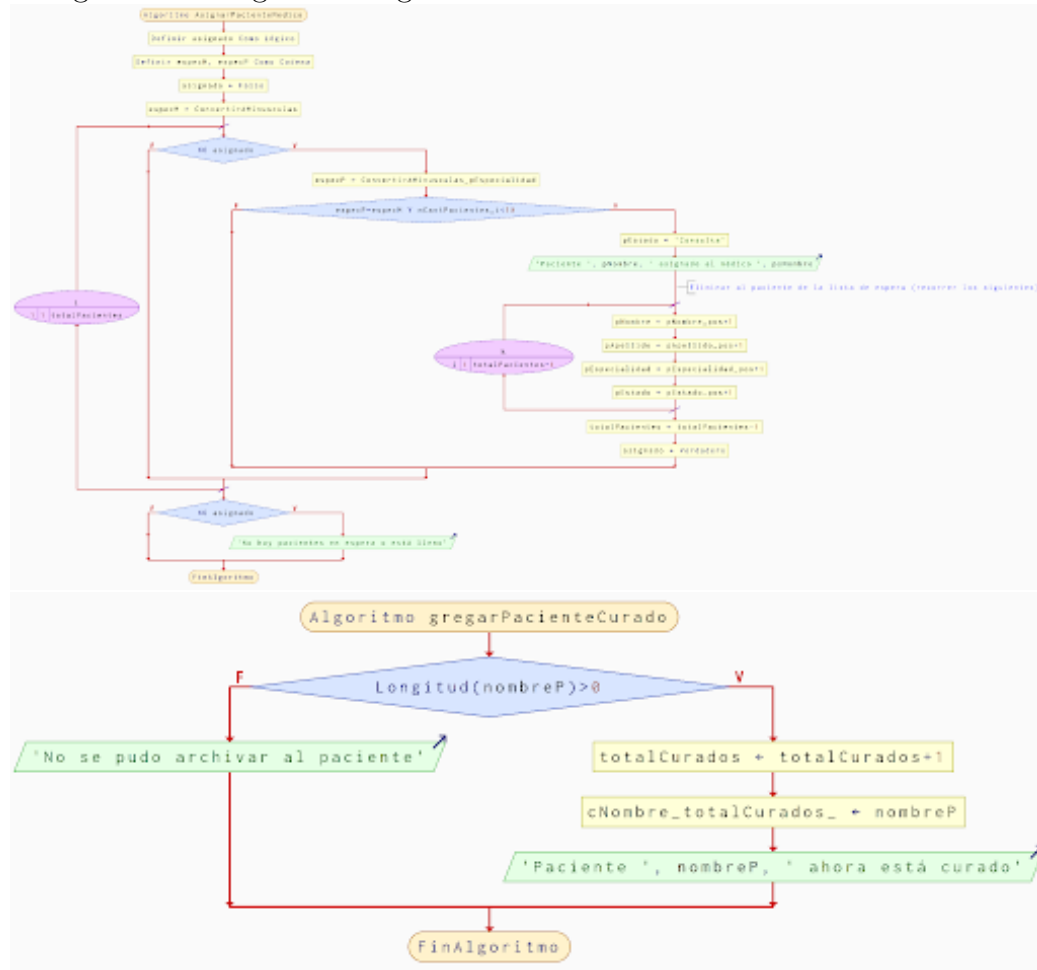
- `asignarPaciente(Medico m)`
- `asignarPaciente(Enfermero e)`

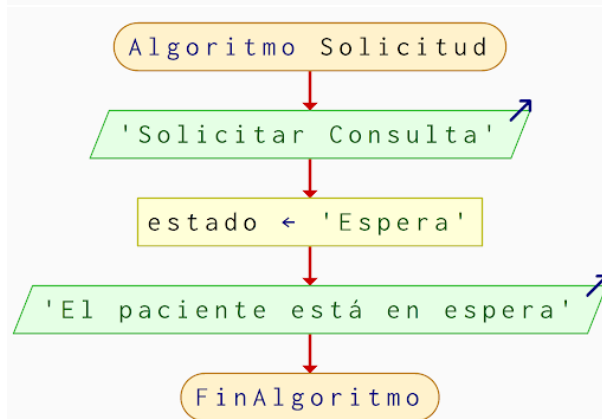
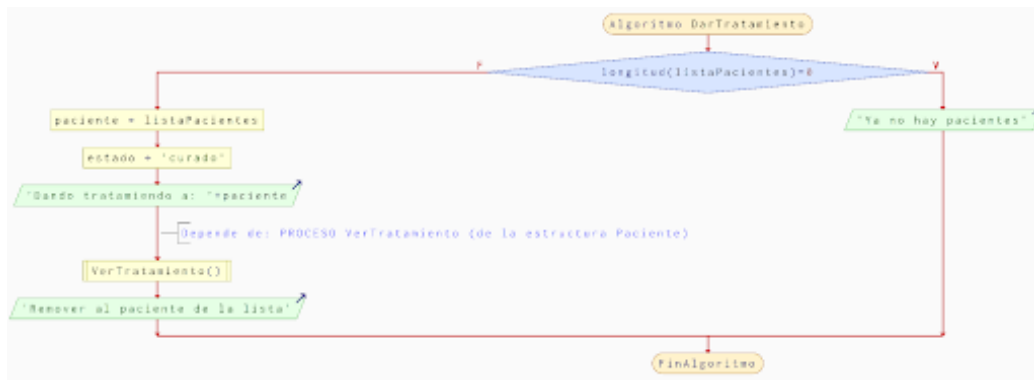
Esto permite que el mismo nombre de método realice operaciones similares adaptadas a diferentes tipos de objetos.

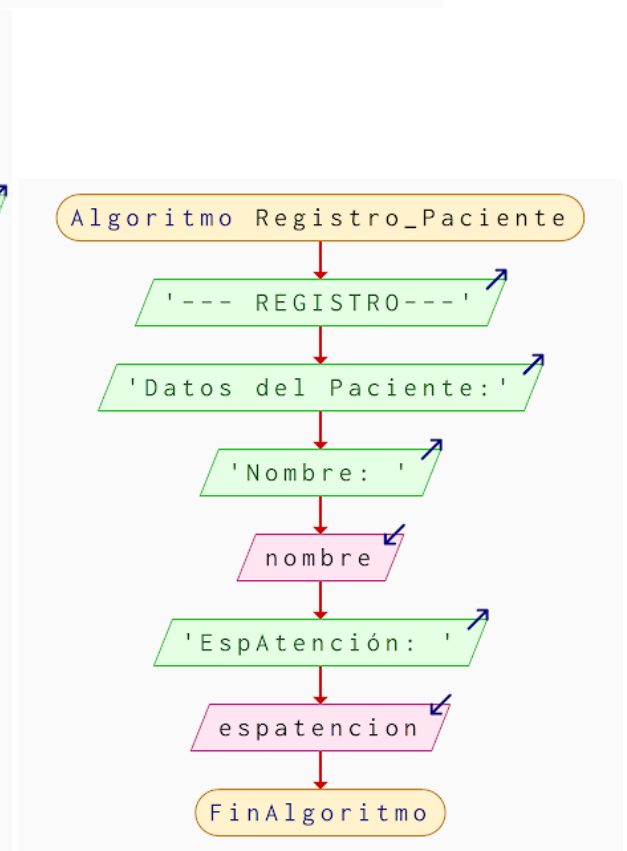
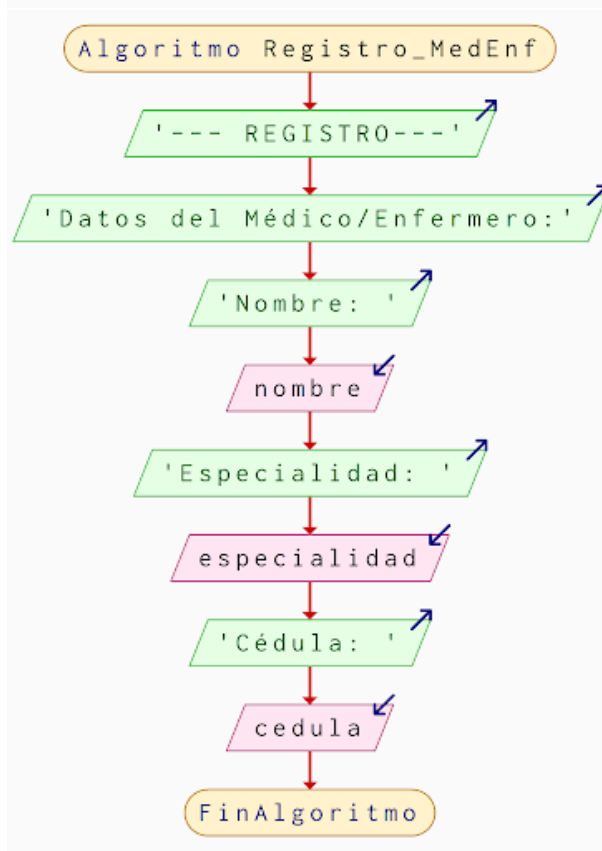
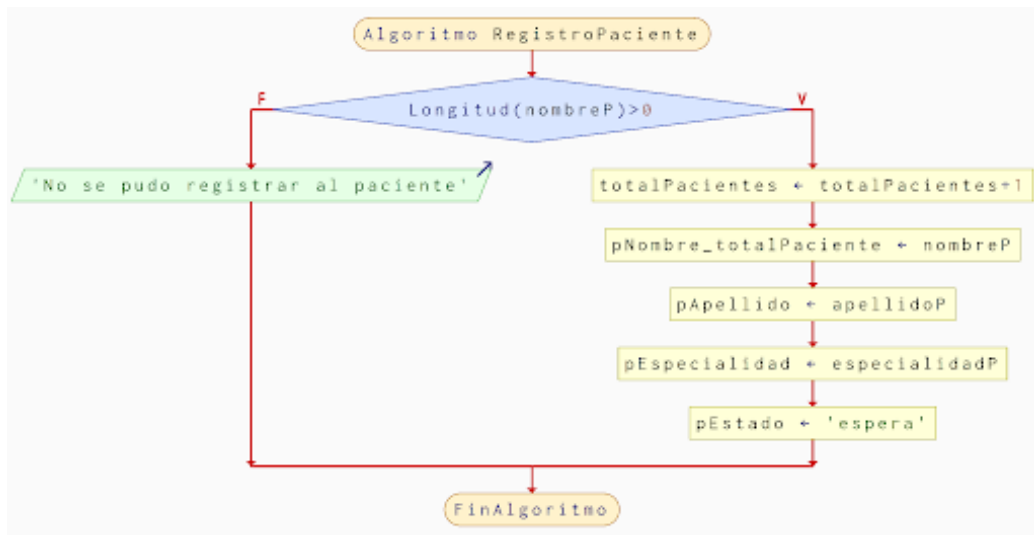
3. Desarrollo

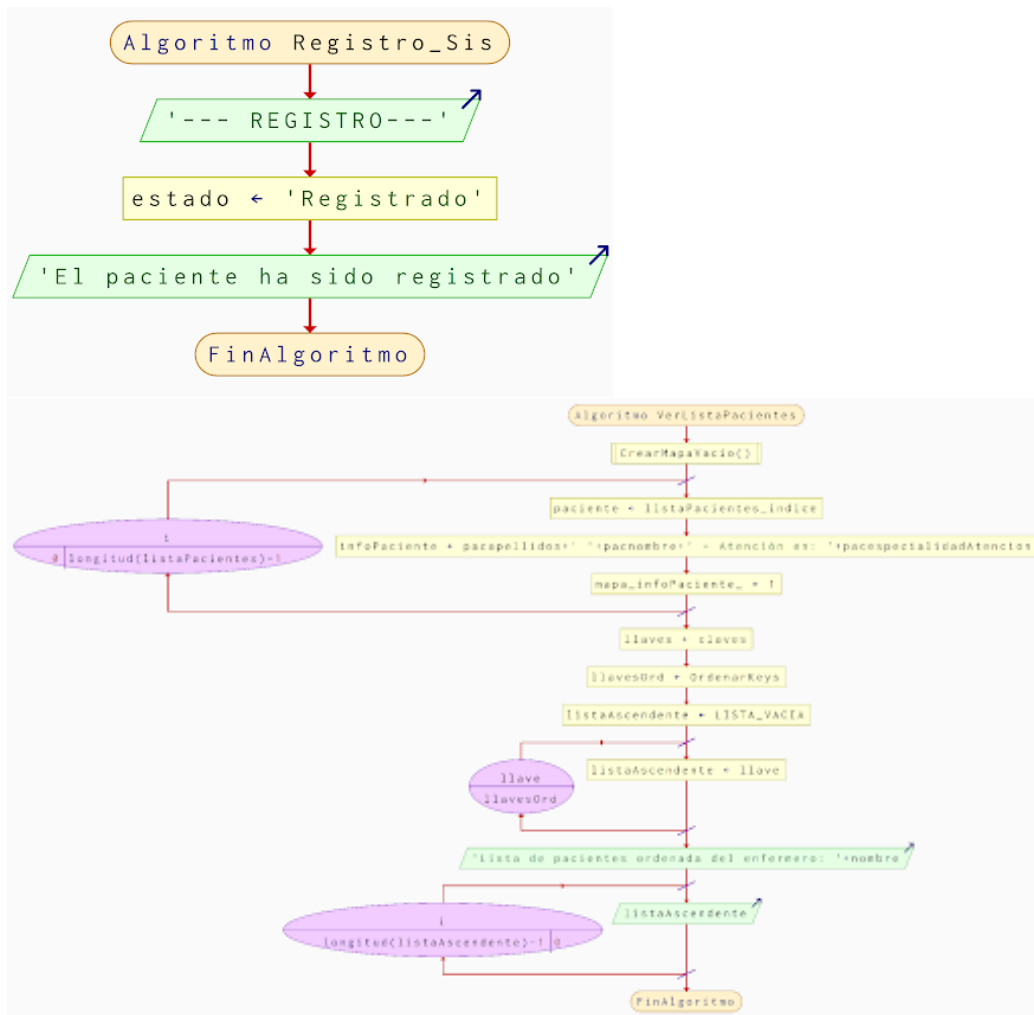
Para el desarrollo del programa, éste se separó en varias clases, donde cada una cumple una función.

Se siguieron los siguientes algoritmos:









Primero, se comenzó con la clase **Paciente**, donde se registran los usuarios que estarán en el hospital. Los atributos de esta clase son: **nombre**, **apellidos**, **especialidadAtencion**, de tipo **String** y **estado** de tipo **char**, ya que es un solo carácter y permite usar el operador **==** directamente, simplificando el código.

El constructor se invoca desde la clase **main**, donde se pide un número de pacientes, y con un ciclo **for** se van llenando los datos de cada uno, y se inicializa el atributo **estado** como 'E', significando "En espera".

Después de establecer el constructor se pasó a la programación de los méto-

dos para clasificar a los pacientes:

Para el método `verTratamiento`, primero se crea una variable local `descripcion` de tipo `String`, y mediante una estructura `else if`, se evalúa el valor almacenado en el atributo `estado`. Si su valor es el carácter 'E', a la variable `descripcion` se le asigna la cadena "En espera"; si su valor es 'R' corresponderá a "Registrado"; si es 'C', a "En consulta"; si es 'T', a "En tratamiento"; y si es 'A', a "Curado". Finalmente el método imprime los datos del paciente y su tratamiento.

El método `registroEnSistema` hace que el atributo `char` cambie de valor a 'R', mientras que `solicitarConsulta` cambia el atributo a 'E'.

Después de `Paciente`, se siguió con la clase `Medico`, la cual se encarga de la administración del personal médico del hospital. Para esta clase se importaron desde `java.util` las interfaces `Map` y `Set`, y las clases `HashMap`, `TreeSet` y `ArrayList`.

Los atributos de esta clase son: `nombre` y `especialidad` de tipo `String`, y `cedula` y `noPacientes` de tipo `int`.

El constructor se invoca desde la clase `main`, donde se pide un número de médicos a registrar, y con un ciclo `for` se van llenando los datos de cada uno, inicializando `noPacientes` en 0. Se usa un `ArrayList` para inicializar una lista dinámica `listaPacientes`, gracias a la clase `ArrayList`, y en ella se guardan los pacientes que se le asignan al médico.

Después de establecer el constructor se pasó a la programación de los métodos:

En el método `registroEnSistema()`, se recibe como parámetro una referencia de tipo `Paciente` llamada `p`, posteriormente se hacen dos validaciones: una verificando que la referencia no esté vacía y otra para verificar que no se exceda el límite de pacientes que el médico puede atender, en este caso 10. Si esto se cumple se añade el paciente a la lista dinámica, y en caso de que alguna no se cumpla se muestra el mensaje "No se encontró al paciente o la lista está llena".

Para el método `solicitarPaciente`, se establece que retorne un objeto de tipo `Paciente`. Primero se verifica que `listaPacientes` no esté vacía; si lo está se muestra el mensaje: "Ya no hay pacientes" y ya no continúa. En caso contrario, se crea una variable local `pacienteTurno` de tipo `Paciente` y se le asigna la referencia del primer elemento en `listaPacientes`. Después se valida el atributo `estado` de `pacienteTurno`; si es igual a 'C', el médico está ocupado y se termina el método. Si el médico está libre se usa el método `darConsulta` usando como parámetro a `pacienteTurno` y finalmente retorna el objeto del paciente después de pasar por el método `darConsulta`.

El método `darConsulta` recibe como parámetro la referencia `p` de tipo `Paciente`, y cambia su atributo `estado` a 'C' si `p` no está vacío; si lo está aparece la leyenda "No se encontró al paciente".

Para el método `darTratamiento` primero se valida con `.isEmpty()` si la lista de pacientes está vacía. En caso de estar vacía se muestra un mensaje y se sale del método; en caso contrario, se verifica que el primer elemento de la lista no tenga como atributo `estado` a 'C'. Si no lo tiene, el método finaliza. En caso contrario, el atributo `estado` pasa a ser 'T', e imprime "El médico indicó tratamiento a:" junto con el nombre del paciente. Se saca al paciente de la fila del médico con `.remove`, y con `this.noPacientes--` se libera un lugar. Finalmente se retorna `p`.

En el método `verListaPacientes()` primero se imprime el mensaje "Lista de pacientes ordenada del médico", seguido del nombre del médico. Posteriormente se revisa con un `if` si la lista está vacía; si lo está, se muestra el mensaje "No hay pacientes que mostrar". En caso contrario se crea una variable `mapa` de tipo `Map`, para asociar datos de tipo `Integer` a datos de tipo `String`. Después, dentro de un ciclo `for`, se usa `get(i)` para obtener el paciente asociado a ese número de la lista. Posteriormente se crea una variable local de tipo `String` llamada `infoPaciente`, que se inserta como llave en el mapa usando `.put()`, colocando el valor 1 como relleno. Después se extrae el conjunto de llaves mediante `.keySet()` y se crea un `TreeSet<String>` para organizar las cadenas automáticamente en orden alfabético ascendente. Posteriormente se pasan a una lista, para finalmente imprimir los pacientes en orden alfabético descendente usando un ciclo `for`.

La clase `Enfermero` en su mayoría es igual a la clase `Medico`, cambiando que

su límite de pacientes es 3 y el método `darTratamiento`.

En el método `darTratamiento`, primero se valida con `.isEmpty()` si la lista de pacientes está vacía. En caso de estar vacía se muestra un mensaje y se sale del método; en caso contrario, se extrae al paciente que está en la posición 0 de la lista usando `get`, y se guarda en `p`. Después se asigna al atributo `estado` de ese paciente la letra 'A', para indicar que se le está dando tratamiento. Se imprime el mensaje "Dando tratamiento" junto con los datos del paciente, y se llama al método `p.verTratamiento` para mostrar su estado. Se saca al paciente de la fila del médico con `.remove`, y con `this.noPacientes--` se libera un lugar. Finalmente se retorna `p`.

Al finalizar la clase `Enfermero`, se pasó a la clase `Sistema`, la cual funciona como administradora del programa. Su función principal es coordinar el registro general del personal (médicos y enfermeros), gestionar la cola de pacientes en espera y controlar los flujos de asignación y alta final. Para esta clase se importaron desde `java.util` la interfaz `Map` y las clases `HashMap` y `ArrayList`.

Los atributos de esta clase son: de tipo `String`, `nombreHospital`; de tipo `int`, `noMedicos`, `noEnfermeros`, `noPacientes`; y se crean las listas `listaMedicos`, `listaEnfermeros`, `listaPacientes` y `listaCurados`. Los atributos de tipo `int` se inician en 0 dentro del constructor y se crean las listas.

Posteriormente se procedió a programar sus métodos:

Para el método `registroMedico`, se usa como parámetro un objeto `m` de tipo `Medico`, y se valida que `m` no sea nulo. Si no lo es, se añade el médico a la lista y se aumenta `noMedicos` en 1. En caso contrario se muestra un mensaje y se sale del método. Los métodos `registroEnfermero` y `registroPaciente` funcionan de la misma manera.

Para el método `asignarPaciente`, se usa como parámetro un objeto `m` de tipo `Medico`. Se verifica si el objeto es nulo, y si lo es se sale del método; en caso contrario, se usa un ciclo `for` que se repite mientras `i` sea menor a la longitud de la lista de pacientes usando `this.listaPacientes.size()`. Dentro de este ciclo se crea un objeto de tipo `Paciente` llamado `p` y se le asigna el paciente extraído de la lista utilizando `get`. Después se crea un `HashMap`

llamado `mapaAux` para guardar cadenas de texto como llave y números enteros de tipo `Integer` como valor. En el `mapaAux` se mete la especialidad que requiere cada paciente mediante la función `.put()`, pero cada especialidad se convierte a minúsculas para asegurar que no haya errores, usando `.toLowerCase()`, y se le pone un 1 como número de relleno en el valor.

Luego, se usa un `if` que evalúa tres condiciones: la primera verifica si la especialidad del médico coincide con la del paciente que guardamos en el mapa; la segunda verifica que el contador `noPacientes` del médico sea menor a 10; y la tercera asegura que el estado del paciente sea la letra "E" de "En espera". Si las tres condiciones se cumplen, se ejecuta el registro en sistema entregando a `p` para que el médico reciba al paciente en su lista personal. Después, se imprime un mensaje con los nombres para avisar que el paciente fue asignado al médico. Luego, el paciente es eliminado de la fila de espera del hospital utilizando `this.listaPacientes.remove(i)` y se le resta 1 a la variable del número de pacientes con `this.noPacientes--`. Por último se usa un `return` para romper la ejecución.

El método `asignarPaciente` para enfermeros funciona en su mayoría igual que para médicos, pero con las reglas de los enfermeros. En las validaciones para asignarle el paciente al enfermero, se valida que las especialidades coincidan, que el paciente esté en tratamiento y que el cupo del enfermero no esté lleno. Si cumple las tres condiciones, le asigna el paciente al enfermero usando `registroEnSistema`, imprime el nombre del paciente y que fue asignado a un enfermero, lo borra de la fila general del hospital mediante el método `remove`, actualiza el contador restando uno a `noPacientes` y termina la ejecución.

Finalmente, en el método `agregarPacienteCurado`, se recibe un objeto `p` de tipo `Paciente`. Se valida que `p` no sea nulo; en caso de serlo imprime el mensaje "No se pudo archivar al paciente" y se sale del método. En caso contrario, se añade el paciente a `listaCurados` y se imprime en un mensaje que el paciente ahora está curado.

Finalmente, se programó la clase `main`, donde se invocan todas las clases anteriores y se ejecuta el programa. Para esta clase se importó desde `java.util` la clase `Scanner`, para leer lo que el usuario ingresa.

Para empezar se usa el método `public static void main`, donde irá todo lo que se programe a continuación. Se crea el objeto `Scanner sc = new Scanner(System.in)` para capturar datos desde el teclado, y el objeto `Sistema hospital = new Sistema()` que funcionará como el núcleo del programa. Al atributo `nombreHospital` se le asigna el texto "Hospital la raza".

Para el registro de médicos se imprime la pregunta de cuántos se van a registrar, y con `sc.nextLine()` se captura lo que ingrese el usuario como cadena, y con el wrapper `Integer` se convierte el texto a `int`. Esto porque si se usa directamente `nextInt`, también guarda el número, pero deja en el buffer un espacio `\n` que luego el siguiente `Scanner` leerá y saltará a la siguiente pregunta.

Posteriormente se abre un ciclo `for` que va desde 0 y se mantiene hasta que `i` sea menor a `numMedicos`. Dentro del ciclo, el programa imprime mensajes para pedir los datos y los guarda en variables temporales: el nombre y especialidad se leen con `sc.nextLine()`, y la cédula se lee con `Integer.parseInt(sc.nextLine())` para evitar que se queden en el buffer el `"\n"`. Ya con los tres datos, se crea un objeto con `Medico m = new Medico(nombre, especialidad, cedula)` y se le pasa como parámetro al método `registroMedico(m)`. Para el registro de enfermeros se sigue la misma lógica, se crea un objeto con `Enfermero e = new Enfermero()` y lo guarda llamando a `registroEnfermero(e)`. También para el registro de pacientes, se crea un objeto `Paciente p = new Paciente()` y antes de meterlo al hospital, se llaman a los métodos `p.registroEnSistema()` y `p.solicitarConsulta()` para inicializar sus variables de estado. Al acabar todos los registros, se usa `sc.close()` e imprime el nombre del hospital.

Posteriormente se procede a asignar pacientes a los médicos. Se abre un ciclo `for` que se mantiene mientras `i` sea menor a `hospital.listaMedicos.size()`. Dentro del ciclo, se extrae al médico actual de la lista `hospital.listaMedicos`. Se declaran dos enteros temporales: `pacientesEnEspera`, al cual se le asigna el número exacto del tamaño que tiene en ese momento la lista de pacientes del hospital, y `pacientesAsignados`, la cual se inicia en 0 para usarse como contador. Después se abre un ciclo `for` que se mantiene mientras `i` sea menor que `pacientesEnEspera`. En este segundo ciclo se crea la variable "antes" que toma el tamaño actual de `hospital.listaPacientes.size()` y justo debajo se ordena la ejecución del método `hospital.asignarPaciente(m)`.

Tras la asignación, con dos `if`, se verifica que `hospital.listaPacientes` sea menor que "antes", para saber si los pacientes en espera redujeron. Si es así se le suma una a `pacientesAsignados`; en caso de que sea igual significa que no fue compatible o ya no hay cupo, por lo que se sale del bucle con `break` para romper el ciclo. Al salir de ese ciclo un último `if` comprueba si `pacientesAsignados == 0`, y si sí imprime que no hay pacientes o que ese médico en específico está lleno.

Posteriormente, se abre un nuevo ciclo `for` dentro de `main` para imprimir el listado ordenado. Luego, se abre un `while` que da vueltas mientras el tamaño de la lista del médico sea mayor a 0 (`m.listaPacientes.size() > 0`). Dentro del `while`, se crea un objeto de clase `Paciente` llamado `pacienteAtendido` que recibe y guarda lo que nos devuelve la función `m.solicitarPaciente()`. Después se invoca a `pacienteAtendido.verTratamiento()` para que el paciente imprima su texto en pantalla. Después, se declara un nuevo objeto `Paciente` `pacienteTratado` el cual recibe el resultado final del método `m.darTratamiento()`. Por último, para que el paciente no se borre del programa, se reingresa a la lista de curados usando:
`hospital.registroPaciente(pacienteTratado).`

Para la asignación de enfermeros, primero se imprime: "Asignación a enfermeros:" y se abre un ciclo `for` que funciona igual que el de médicos, pero con las reglas de enfermeros.

Para la parte final se imprime "Tratamiento de enfermeros:", después se abre un ciclo `for` y va recorriendo el listado de enfermeros y extrayéndolos para mandar a llamar a `e.verListaPacientes()`. Luego abre otro ciclo `while` que ejecutará el código mientras que las listas del enfermero tengan elementos (`e.listaPacientes.size() > 0`). Dentro de este `while`, se declara un objeto `Paciente` `pacienteCurado` que recoge lo que le retorna el método `e.darTratamiento()` y ese paciente es pasado a la lista usando:
`hospital.agregarPacienteCurado(pacienteCurado).`

Cuando todos los ciclos terminan, el `Main` imprime el texto "Total de pacientes curados archivados:" y le concatena la cantidad que arroja
`hospital.listaCurados.size()`.

4. Resultados

```
● lesshernandez@MacBook-Air-de-LESS ProyectoPoo % javac Main.java Enfermero.java Medico.java
  Paciente.java Sistema.java
● lesshernandez@MacBook-Air-de-LESS ProyectoPoo % java Main.java
```

Compilando y ejecutando de esta forma para que Java pueda compilar todos los archivos .java al mismo tiempo y generando un archivo .class para cada uno.

```
¿Cuántos médicos vas a registrar?: 4
Datos del Médico 1:
Nombre: Luis Hernández
Especialidad: Pediatría
Cédula: 1111
Datos del Médico 2:
Nombre: Diego Cedillo
Especialidad: Dermatología
Cédula: 2222
Datos del Médico 3:
Nombre: Ana López
Especialidad: Cardiología
Cédula: 4444
Datos del Médico 4:
Nombre: Sofía Jimenez
Especialidad: Dermatología
Cédula: 3333
Registro de enfermeros:
¿Cuántos enfermeros vas a registrar? 3
Datos del Enfermero 1:
Nombre: Jose Angel López
Especialidad: Dermatología
Cédula: 5555
Datos del Enfermero 2:
Nombre: Mariana Puebla
Especialidad: Cardiología
Cédula: 6666
Datos del Enfermero 3:
Nombre: Juan Pérez
Especialidad: Pediatría
Cédula: 7777
Registro de pacientes:
¿Cuántos pacientes vas a registrar? 5
Datos del Paciente 1:
Nombre: Luis
Apellidos: Hernández Cabrera
Especialidad que requiere: Pediatría
paciente registrado
Datos del Paciente 2:
Nombre: Jesus
Apellidos: Contreras López
Especialidad que requiere: Dermatología
paciente registrado
Datos del Paciente 3:
Nombre: Estefanía
Apellidos: Ocaña García
Especialidad que requiere: Cardiología
paciente registrado
Datos del Paciente 4:
Nombre: Romina
Apellidos: Herrera Ocampo
Especialidad que requiere: Dermatología
paciente registrado
Datos del Paciente 5:
Nombre: Alex
Apellidos: Sierra Toledo
Especialidad que requiere: Cardiología
paciente registrado
```

Iniciamos registrando el número de médicos y los datos de cada uno, después generamos el registro de los enfermeros con su respectivos datos y finalmente los datos de cada paciente , haciendo énfasis en la especialidad que requiere y en sus apellidos ya que posteriormente serán importantes.

```
Hospital: Hospital La Raza
```

```
Asignación a médicos:
```

```
Paciente Luis asignado al médico Luis Hernández
```

```
No hay pacientes en espera o está lleno para el médico: Diego Cedillo
```

```
Paciente Estefanía asignado al médico Ana López
```

```
Paciente Alex asignado al médico Ana López
```

```
Paciente Jesus asignado al médico Sofía Jimenez
```

```
Paciente Romina asignado al médico Sofía Jimenez
```

Se asignan los pacientes correspondientes a los médicos de acuerdo a su especialidad, en este caso Diego Cedillo no tiene asignaciones ya que hay otro médico con su misma especialidad.

```
Asignación médica:
```

```
Lista de pacientes ordenada del médico :Luis Hernández
```

```
Hernández Cabrera Luis - Especialidad: Pediatría (0)
```

```
El nombre del paciente es: Luis Hernández Cabrera y su tratamiento actual es: En consulta
```

```
El médico indicó tratamiento a: Luis
```

```
Lista de pacientes ordenada del médico :Diego Cedillo
```

```
No hay pacientes que mostrar
```

```
Lista de pacientes ordenada del médico :Ana López
```

```
Sierra Toledo Alex - Especialidad: Cardiología (1)
```

```
Ocaña García Estefanía - Especialidad: Cardiología (0)
```

```
El nombre del paciente es: Estefanía Ocaña García y su tratamiento actual es: En consulta
```

```
El médico indicó tratamiento a: Estefanía
```

```
El nombre del paciente es: Alex Sierra Toledo y su tratamiento actual es: En consulta
```

```
El médico indicó tratamiento a: Alex
```

```
Lista de pacientes ordenada del médico :Sofía Jimenez
```

```
Herrera Ocampo Romina - Especialidad: Dermatología (1)
```

```
Contreras López Jesus - Especialidad: Dermatología (0)
```

```
El nombre del paciente es: Jesus Contreras López y su tratamiento actual es: En consulta
```

```
El médico indicó tratamiento a: Jesus
```

```
El nombre del paciente es: Romina Herrera Ocampo y su tratamiento actual es: En consulta
```

```
El médico indicó tratamiento a: Romina
```

Siguiendo con el programa , en la asignación médica se enlistan los pacientes de cada médico con sus apellidos de forma alfabética descendente (Z-A) , muestra la especialidad que requieren y si hay más pacientes con (0) y (1) podemos distinguir el turno de cada uno.

Después muestra su tratamiento:

```

Asignación a enfermeros:
Paciente Jesus asignado al enfermero Jose Angel López
Paciente Romina asignado al enfermero Jose Angel López
Paciente Estefanía asignado al enfermero Mariana Puebla
Paciente Alex asignado al enfermero Mariana Puebla
Paciente Luis asignado al enfermero Juan Pérez

Tratamiento de enfermeros:

Lista de pacientes ordenada del enfermero :Jose Angel López
Herrera Ocampo Romina - Especialidad: Dermatología (1)
Contreras López Jesus - Especialidad: Dermatología (0)
Dando tratamiento a: Jesus
El nombre del paciente es: Jesus Contreras López y su tratamiento actual es: Curado
Paciente Jesus ahora está curado
Dando tratamiento a: Romina
El nombre del paciente es: Romina Herrera Ocampo y su tratamiento actual es: Curado
Paciente Romina ahora está curado

Lista de pacientes ordenada del enfermero :Mariana Puebla
Sierra Toledo Alex - Especialidad: Cardiología (1)
Ocaña García Estefanía - Especialidad: Cardiología (0)
Dando tratamiento a: Estefanía
El nombre del paciente es: Estefanía Ocaña García y su tratamiento actual es: Curado
Paciente Estefanía ahora está curado
Dando tratamiento a: Alex
El nombre del paciente es: Alex Sierra Toledo y su tratamiento actual es: Curado
Paciente Alex ahora está curado

Lista de pacientes ordenada del enfermero :Juan Pérez
Hernández Cabrera Luis - Especialidad: Pediatría (0)
Dando tratamiento a: Luis
El nombre del paciente es: Luis Hernández Cabrera y su tratamiento actual es: Curado
Paciente Luis ahora está curado

```

Ahora viene la asignación de enfermeros que es similar a la de los médicos dependiendo de su especialidad , igualmente la lista de los pacientes para los enfermeros funciona igual que con los doctores sin embargo los enfermeros pueden atender hasta 3 personas a la vez.

```
Total de pacientes curados archivados: 5
```

Finalmente se muestra el total de pacientes curados.

5. Conclusiones

En esta primera etapa del proyecto se logró modelar el funcionamiento de un sistema hospitalario mediante el uso de clases y objetos, cumpliendo con los requisitos en la especificación.

Se implementaron las clases Sistema, Médico, Enfermero y Paciente, cada una con sus atributos y comportamientos correspondientes y permitiendo simular el registro de personal médico y pacientes, así como la asignación de cada uno de estos con base en la especialidad requerida y la disponibilidad de cupo (10 pacientes por médico y 3 por enfermero).

Se utilizó una estructura lineal (ArrayList) para el manejo de las listas de médicos, enfermeros y pacientes y poder usar las operaciones de registro, búsqueda y asignación. Asimismo, se implementó el método sobrecargado asignarPaciente, para asignar a un médico o a un enfermero, y se estableció un mecanismo mediante el atributo estado para determinar en qué fase se encuentra cada paciente (espera, consulta o tratamiento).

Con esta etapa se sentaron las bases estructurales del sistema, dejando preparado el diseño de clases para incorporar, en etapas posteriores, funcionalidades más complejas del simulador hospitalario, así como una interfaz de interacción más completa para el usuario.

Referencias

- [1] C. Álvarez Caules. *Java forEach y sus opciones*. <https://www.arquitecturajava.com/java-foreach-y-sus-opciones/>. Publicado: 25 de junio de 2021, Accedido: 22 de septiembre de 2026.
- [2] Portal Académico CCH-UNAM. *Constructores - Principios de Programación Orientada a Objetos*. <https://portalacademico.cch.unam.mx/cibernetica2/principios-programacion-orientada-a-objetos/constructores>. Accedido: 20 de septiembre de 2026.
- [3] Ministerio de Educación y FP. 6.4.- *Librerías Java*. — *PROG03. Contenidos Unidad 3*. https://sarreplec.caib.es/pluginfile.php/10221/mod_resource/content/3/64_librerias_java.html. Accedido: 22 de septiembre de 2026.
- [4] GeeksforGeeks. *ArrayList in Java*. <https://www.geeksforgeeks.org/arraylist-in-java/>. Accedido: 21 de septiembre de 2026.
- [5] GeeksforGeeks. *HashMap in Java*. <https://www.geeksforgeeks.org/java/java-util-hashmap-in-java-with-examples/>. Accedido: 21 de septiembre de 2026.
- [6] GeeksforGeeks. *TreeSet in Java*. <https://www.geeksforgeeks.org/treeset-in-java-with-examples/>. Accedido: 21 de septiembre de 2026.
- [7] Oracle. *Classes and Objects*. <https://docs.oracle.com/javase/tutorial/java/concepts/>. Accedido: 20 de septiembre de 2026.
- [8] Throphic. *Estructuras de control en Java*. https://dat-science.com/estructuras-de-control-en-java/#Estructura_if. Accedido: 23 de septiembre de 2026.
- [9] W3schools.com. *Java User Input (Scanner class)*. https://www.w3schools.com/java/java_user_input.asp. Accedido: 11 de septiembre de 2026.